

ContextPilot: Teaching Agents for Proactive Context Management via Fine-grained RL

Zhuoshi Pan^{1,2}♣, Qizhi Pei³♣, Junru Lu², Honglin Lin³

H. Vicky Zhao¹♡, Di Yin², Xing Sun²♡

¹Tsinghua University ²Tencent Youtu Lab ³Shanghai AI Lab

♣ Equal Contribution; ♡ Corresponding Authors.

pzs23@mails.tsinghua.edu.cn; vzhao@tsinghua.edu.cn; winfredsun@tencent.com

Abstract

Long-horizon agentic tasks require large language models (LLMs) to iteratively retrieve, integrate, and maintain dispersed information across multi-turn interactions, but preserving all interaction histories leads to a continuously growing working context. Recent proactive context management methods allow models to edit their own working context with specialized tools, yet they still face **three key limitations**: (1) a *limited toolset* restricted to search, deletion, and summarization, with no support for global planning, long-term memory, and adaptive compression; (2) *inefficient exploration* that treats context management actions uniformly despite their heterogeneous impacts on final outcomes; and (3) *coarse-grained credit assignment* that assigns the final trajectory-level reward to all intermediate context editing actions during RL. To bridge these gaps, we introduce **ContextPilot**, a proactive context management framework for long-horizon agentic reasoning. Our approach systematically augments the toolset with planning, long-term memory, and soft context offloading tools. We further propose an RL method tailored for context management, which uses context and entropy variation to identify critical context control actions for branch sampling and estimates action-level advantages from all branched trajectories that pass through these actions. Experiments on long-context QA and deep search tasks show that **ContextPilot** achieves stronger performance with a more compact working context, outperforming existing baselines across various base models and benchmarks. Code is available at this [URL](#).

1 Introduction

Large language models (LLMs) have shown strong reasoning capabilities on information-intensive tasks, including long-context QA (Zhang et al., 2024; Du et al., 2025) and deep search (Wei et al., 2025; Team et al., 2025). These tasks require

models to identify key evidence from long contexts, integrate complex relations, and, when necessary, seek information through multi-turn tool interactions (Feng et al., 2025). Prevailing agentic paradigms like ReAct (Yao et al., 2023) typically append prior reasoning, tool calls and tool responses to the context as the interaction proceeds, resulting in a rapidly growing context. Prior studies (Shao et al., 2025; Wu et al., 2025b; Lu et al., 2025; Su et al., 2026) address context growth through *human-designed workflows*, where truncation or summarization is triggered by fixed rules. Although straightforward, they give the model no control over its own context, and are difficult to adapt to different scenarios (Liu et al., 2026b). To alleviate this, recent works (Li et al., 2025b; Ye et al., 2025; Liu et al., 2026b) propose **proactive context management**, allowing models to spontaneously manage their working context through context editing tools. Despite its flexibility, this paradigm still faces three key limitations.

First, existing context management toolsets, which typically consist of search, deletion, and summarization, are insufficient for managing the context of long-horizon tasks. Effective context management requires more than simply removing or summarizing content: models also need to build long-term memory across scattered fragments (Chhikara et al., 2025; Xu et al., 2026), maintain structured entity-event episodes (Lu et al., 2026), and plan globally before taking subsequent actions (Yao et al., 2023; Liu et al., 2026a). **Second**, although recent studies (Yu et al., 2026; Liu et al., 2026b) have applied RL fine-tuning to help models further familiarize with context editing tools, *their training procedures are not specifically adapted for context management*. Unlike common tools, context editing tools can substantially overwrite the interaction history and exert a larger influence on subsequent steps (Shao et al., 2025). As shown in Figure 2, trajectory branches originating from

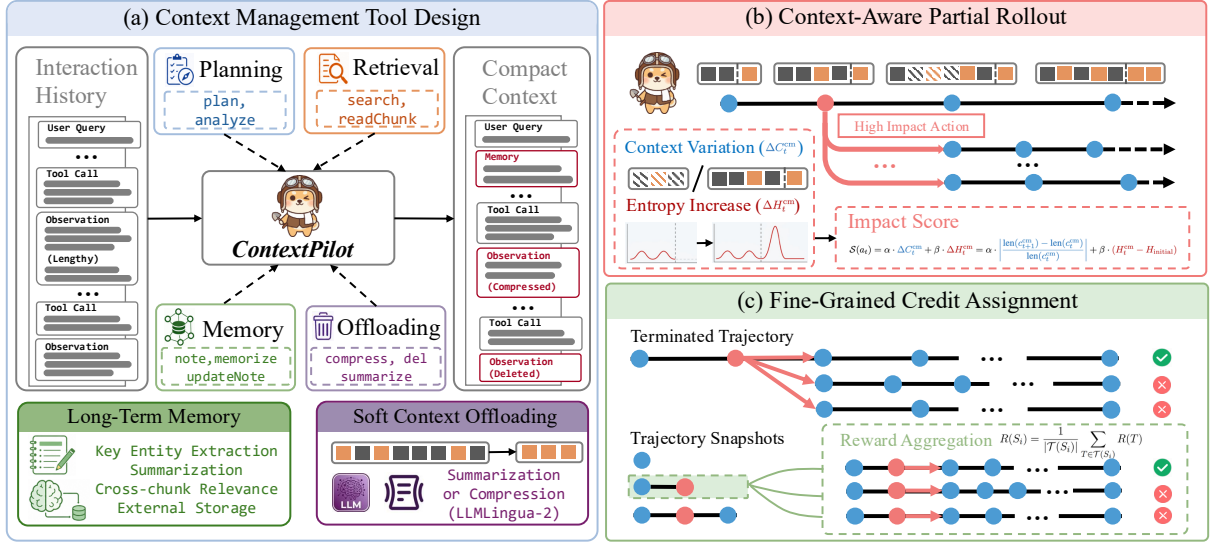


Figure 1: Overview of **ContextPilot**: (a) Extending the context management tools to support planning, long-term memory, and soft context offloading; (b) Improved RL training algorithm with context-aware partial rollout and (c) fine-grained snapshot-level credit assignment.

different context management operations exhibit substantially different variance in their final success rates, indicating that certain operations are more sensitive than others. Unfortunately, traditional RL relies on trajectory-level rollouts (Dong et al., 2025), limiting adaptive exploration for various context management decisions. **Third**, existing training procedures directly assign the final trajectory-level reward to all intermediate context management actions, neglecting fine-grained credit assignment (Li et al., 2026).

To address these challenges, we propose **ContextPilot**, a proactive context management framework for long-horizon agentic reasoning. Our contributions are threefold: (1) On the **tool** side, our framework extends the existing **context management toolset** with planning, long-term memory, and soft context offloading tools, enabling the agent to better control its context across long-horizon interactions. (2) On the **training** side, we design an RL training method tailored to context management. We propose **context-aware partial rollout**, which uses context and entropy variation to identify critical context management actions for branch sampling. For credit assignment, we incorporate rewards from all subsequent branches to estimate the advantage of an intermediate context management action, yielding a more **fine-grained reward signal**. (3) On the **evaluation** side, whereas prior work mostly focuses on long-context QA, we extend the evaluation to deep search tasks. Experiments show that **ContextPilot** maintains a more

compact context while achieving superior performance, bringing consistent gains and surpassing existing baselines.

2 Related Work

2.1 Passive Context Management

To handle growing interaction histories, existing methods typically manage historical context through predefined rules, such as truncating or summarizing messages once the context exceeds a length threshold. Some methods (Lu et al., 2023; Pan et al., 2025; Wu et al., 2025b; Su et al., 2026) are training-free and rely on fixed workflows or external modules to summarize, compress, or extract key information from historical context. Another line of work (Lu et al., 2025; Shao et al., 2025) incorporates context folding into RL training, enabling models to summarize context and resume reasoning from the compressed state. Although these methods reduce context length, the model remains a passive recipient of rule-managed context rather than an active manager that can decide *when* and *how* to manage its working context.

2.2 Agentic Proactive Context Management

Beyond the passive paradigm, recent work equips agents with context management tools, allowing agents to actively decide *when* and *how* to edit their working context. MemGPT (Packer et al., 2024) is an early framework that treats an agent’s memory as virtual memory and manages it through tool calls. Yu et al. (2026) and Xu et al. (2026) propose

memory editing tools, training the agent to control its memory without an external controller. Sculptor (Li et al., 2025b), StateLM (Liu et al., 2026b), and MemAct (Zhang et al., 2025) provide models with context editing tools such as fragmentation, search, summarization, and deletion, and fine-tune models to use these tools. AgentFold (Ye et al., 2025) uses SFT to learn multi-scale folding operations that can condense the history. Despite demonstrating the promise of proactive context management, existing methods are still constrained by a narrow set of context management tools, and their training procedures fail to consider the varying impact of these tools. In contrast, **ContextPilot** improves proactive context management by enriching tool design and refining training paradigms.

3 Preliminary

Before introducing **ContextPilot**, we first revisit the concepts of context management in agentic reasoning and conduct a pilot study.

3.1 From Passive to Proactive: Context Management in Agentic Reasoning

In prevailing agentic reasoning paradigms like ReAct (Yao et al., 2023), the agent can perform reasoning and invoke tools to interact with an environment $\mathcal{E}$. Formally, given the user query $c_0 = q$, at the i -th step, the LLM π_θ generates thought t_i and tool call a_i based on the current context c_i :

$$(t_i, a_i) \sim \pi_\theta(\cdot \mid c_i). \quad (1)$$

In the next step, the generated thought t_i , tool call a_i and the environment feedback $o_i \sim \mathcal{E}(\cdot \mid a_i)$ are appended to the context $c_{i+1} = c_i \oplus (t_i, a_i, o_i)$. The model then iterates this cycle until accomplishing the task. As agentic reasoning proceeds, the context grows monotonically. To mitigate the context overload, **proactive context management** enables models to manage their context through specific tools, such as search, retrieval, deletion, and summarization. Specifically, let $\mathcal{A}$ denote the set of available actions, and $\mathcal{A}_{\text{cm}} \subseteq \mathcal{A}$ denote the subset of context management actions. Among them, we distinguish context editing actions as $\mathcal{A}_{\text{ce}} \subseteq \mathcal{A}_{\text{cm}}$, which modify the previous messages. At step i , when the model invokes a context management action $a_i^{\text{cm}} \in \mathcal{A}_{\text{cm}}$, its context is updated by a context transition function $\mathcal{F}$:

$$c_{i+1} = \mathcal{F}(c_i, a_i^{\text{cm}}) \oplus (t_i, a_i, o_i). \quad (2)$$

Under this paradigm, training solely on the final trajectory is inadequate, as context editing operations (i.e., $a_i^{\text{ce}} \in \mathcal{A}_{\text{ce}}$) will modify historical messages. To address this, existing studies (Li et al., 2025b; Liu et al., 2026b) adopt *trajectory snapshots* and *token-level loss masking*. Suppose a trajectory T contains K tool invocations $\{a_i\}_{i=1}^K$, of which M are context editing operations. These context editing operations segment the multi-step trajectory into $M + 1$ independent trajectory snapshots $\mathcal{S} = \{S_1, S_2, \dots, S_{M+1}\}$. Each snapshot is then treated as an independent training instance, ensuring that every intermediate context state is incorporated in training. Furthermore, token-level loss masking is applied to exclude the loss of outputs that have already appeared in previous trajectory snapshots. This prevents redundant optimization over earlier tool calls and improves training stability and efficiency.

3.2 Pilot Study

To better understand the impact of context management tools, we branch from context management actions in existing trajectories, sample continuation trajectories, and compute the standard deviation of their final success rates. As shown in Figure 2, this variance differs substantially across actions, suggesting that certain context management actions have a larger impact on the final outcomes and should receive more exploration budget. For credit assignment, Figure 3 shows that final correctness can be misaligned with context management quality: a correct trajectory may rely on repeated retrieval, whereas an incorrect one may still perform reasonable management. Therefore, directly assigning the final trajectory-level reward to all intermediate snapshots may reinforce inefficient context management behaviors and erroneously penalize reasonable decisions, motivating fine-grained action-level rewards.

4 Methodology

In this section, we elaborate on the methodology of **ContextPilot**. We first describe the design of the extended toolset and SFT data construction, followed by the reinforcement learning recipe.

4.1 Context Management Toolset Design

ContextPilot builds upon the basic toolset of StateLM (Liu et al., 2026b) and introduces additional tools for planning, long-term memory

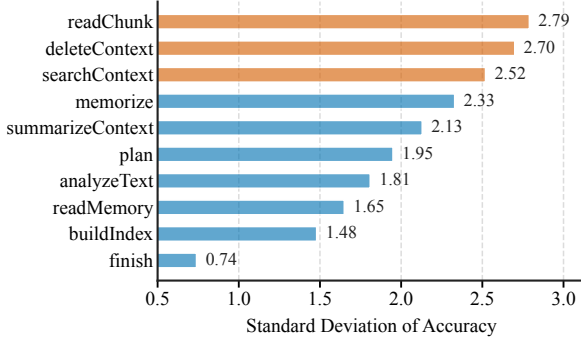


Figure 2: Impact of different tools on final outcomes: for each tool, we branch from it, sample 10 continuation rollouts, and report the standard deviation of their task success rates. Results are evaluated with Qwen3-8B on NovelQA (Wang et al., 2025).

Correct Answer Weak Context Management Q: Name the reviews editor of a mid-20th-century university journal, given an article they wrote and their Dec-2023 role. Gold: Robert D. McBain Answer: Robert D. McBain ✓ Tool chain: analyze → index → search×3 → readChunk → search×2 → readChunk → ... → readChunk → search×2 → readChunk → finish	Wrong Answer Strong Context Management Q: Find the guitarist who, in 70s high school, founded a band known for conscious multilingual music, and died in 2003. Gold: Frank Leepa Answer: Ty Longley ✗ Tool chain: analyze → index → search → readChunk → note → delete×2 → search×2 → readChunk → updateNote → delete×2 → readNote → finish
---	--

Figure 3: Case study on the mismatch between task correctness and context management quality. Cases are sampled from the traces of StateLM-8B (Liu et al., 2026b) on BrowseComp+ (Chen et al., 2025). Questions are condensed to save space.

and soft context offloading, as detailed in Table 1. To support long-term memory, we introduce `memorize`, which extracts structured information about entities, timestamps, and event episodes, and builds edges among related memory items. The model can later call `readMemory` to retrieve a target memory with its neighbors. For context offloading, we introduce soft-deletion tools, including `summarizeContext`, `compressContext`, and `foldHistory`. `foldHistory` condenses all historical messages into some keywords and a summary. The folded history can later be recovered by calling `searchContext`, thus preserving reversibility while making current context concise. We treat context offloading and memory-writing tools as context editing tools, and segment trajectories into snapshots at these operations because they modify the interaction history. Through these extensions, *ContextPilot* provides a comprehensive context management toolset, empowering long-horizon agents to effectively manage their working context.

Category	Tool Name	Function Description
Perception & Planning	analyzeText	Calculate context length.
	checkBudget	Check remaining token budget.
Information Retrieval	plan	Propose a concise plan.
	buildIndex	Build a searchable index.
	searchContext	Search for relevant content.
	readChunk	Load a specific context chunk.
Memory Management	readMultiChunks	Batch load multiple context chunks.
	note	Record key information into a note.
	updateNote	Update the content of a note.
	readNote	Load the content of a note.
	memorize	Extract key information and cross-chunk relations into event memory.
	updateMemory	Update a memory item.
Context Offloading	readMemory	Load a specific memory item.
	deleteContext	Replace a message with a placeholder.
	summarizeContext	Replace a message with a summary.
	compressContext	Compress a message via a lightweight compression model (Pan et al., 2024).
	foldHistory	Discard all historical messages and build a searchable index.

Table 1: Context management tools of *ContextPilot*. Tools highlighted in purple are newly introduced.

4.2 Supervised Fine-Tuning Data Synthesis

Before constructing SFT data, we design a *context management harness* based on the developed toolset. Through carefully orchestrated rules, the harness provides the teacher model with hints and constraints at appropriate steps. For example, it guides the teacher to inspect retrieved content with `readChunk` after `search`. When the context length exceeds a predefined threshold, it restricts the model to context offloading tools for cleaning up. These hints and constraints are used only as generation-time scaffolding and are excluded from the final SFT trajectories. Additional details of SFT data construction are in Appendix C.

4.3 RL Training with Partial Rollout and Fine-Grained Credit Assignment

Algorithm 1 summarizes the complete RL training procedure, combining context-aware partial rollout with snapshot-level credit assignment.

Context-Aware Partial Rollout. Inspired by ARPO (Dong et al., 2025), we introduce partial rollout, which allocates **more exploration budget** to those critical context management decisions. To identify these critical decisions, we first calculate the context variation:

$$\Delta C_t^{\text{cm}} = \left| \frac{\ln(c_{t+1}^{\text{cm}}) - \ln(c_t^{\text{cm}})}{\ln(c_t^{\text{cm}})} \right|, \quad (3)$$

and the entropy variation:

$$\Delta H_t^{\text{cm}} = H_t^{\text{cm}} - H_{\text{initial}}, \quad (4)$$

for each context management operation $a_t^{\text{cm}} \in \mathcal{A}_{\text{cm}}$. H_t^{cm} is the average entropy of k generated

tokens after receiving the observation at step t :

$$H_t^{\text{cm}} = \frac{1}{k} \sum_{i=0}^{k-1} \left(- \sum_{j=1}^V p_{t+i,j} \log p_{t+i,j} \right), \quad (5)$$

where V is the vocabulary size, $p_{t+i,j}$ denotes the probability after softmax and H_{initial} is the average entropy of the first k tokens at the beginning of the trajectory. We use the initial entropy H_{initial} instead of the entropy of the preceding step H_{t-1}^{cm} as the reference, because partial rollout aims to identify critical tool calls that induce significant uncertainty changes relative to the initial query state, instead of local fluctuations between adjacent steps. Based on these metrics, the sensitivity score $\mathcal{S}(a_t^{\text{cm}})$ can be defined as:

$$\mathcal{S}(a_t^{\text{cm}}) = \alpha \cdot \Delta C_t^{\text{cm}} + \beta \cdot \Delta H_t^{\text{cm}}, \quad (6)$$

where α and β balance the weights of the two metrics. Appendix L provides a sensitivity analysis of the weight α . During the rollout, given a global rollout budget of N snapshots per query, we first perform trajectory-level rollouts and segment the resulting trajectories into M snapshots at context management actions. If $M < N$, the remaining snapshot budget is allocated to partial rollouts: we rank all context management actions by their sensitivity score $\mathcal{S}(a_t^{\text{cm}})$ in descending order and select the top $N - M$ actions as branching points. Starting from the parent node of each selected action, we sample additional sub-trajectories to enrich the snapshot pool. This adaptive partial rollout mechanism facilitates more exploration at critical context management actions that have a larger impact.

Fine-Grained Credit Assignment. Benefiting from the partial rollout mechanism, we can assign credit to intermediate trajectory snapshots in a more fine-grained manner. Specifically, suppose a complete terminal trajectory T is segmented into M trajectory snapshots $\{S_1, S_2, \dots, S_M\}$ at context editing operations. Unlike existing works (Li et al., 2025b; Liu et al., 2026b) that directly assign the final sparse reward $R(T)$ to all intermediate snapshots, we estimate the value of each snapshot using the reward of all its subsequent branches. **For a terminal snapshot** $S_M = T$, the reward includes an outcome reward R_{out} , a format reward R_{fmt} , and a penalty item R_{pen} :

$$R(S_M) = R_{\text{out}} + R_{\text{fmt}} + R_{\text{pen}}. \quad (7)$$

Algorithm 1 RL Training with Context-Aware Partial Rollout and Fine-Grained Credit Assignment

Require: Initial policy π_θ , queries $\mathcal{D}$, snapshot budget N , initial rollout number G , GRPO update epochs μ , context management actions $\mathcal{A}_{\text{cm}}$, context editing actions $\mathcal{A}_{\text{ce}}$

Ensure: Updated policy π_θ

```

1: for each RL iteration do
2:   Sample a query batch  $\mathcal{B} \subset \mathcal{D}$  and set  $\pi_{\theta_{\text{old}}} \leftarrow \pi_\theta$ 
3:   for each query  $q \in \mathcal{B}$  do
4:     Sample initial trajectories  $\mathcal{T}_q^{\text{init}}$  with  $|\mathcal{T}_q^{\text{init}}| = G$ 
5:     Segment  $\mathcal{T}_q^{\text{init}}$  at context editing actions to obtain snapshots  $\mathcal{V}_q^{\text{init}}$ 
6:     Compute  $\mathcal{S}(a_t^{\text{cm}})$  for each  $a_t^{\text{cm}} \in \mathcal{A}_{\text{cm}}$  in  $\mathcal{T}_q^{\text{init}}$  by Eq. 6
7:     Initialize  $\mathcal{T}_q^{\text{part}} \leftarrow \emptyset$  and  $\mathcal{V}_q^{\text{part}} \leftarrow \emptyset$ 
8:     if  $|\mathcal{V}_q^{\text{init}}| < N$  then
9:       Select the top  $N - |\mathcal{V}_q^{\text{init}}|$  context management actions by sensitivity score
10:      Branch from selected actions to sample partial-rollout trajectories  $\mathcal{T}_q^{\text{part}}$ 
11:      Segment  $\mathcal{T}_q^{\text{part}}$  at context editing actions to obtain snapshots  $\mathcal{V}_q^{\text{part}}$ 
12:    end if
13:     $\mathcal{T}_q \leftarrow \mathcal{T}_q^{\text{init}} \cup \mathcal{T}_q^{\text{part}}, \mathcal{V}_q \leftarrow \mathcal{V}_q^{\text{init}} \cup \mathcal{V}_q^{\text{part}}$ 
14:    for each snapshot  $S_i \in \mathcal{V}_q$  do
15:      Initialize  $R_{\text{acc}}(S_i) \leftarrow 0$  and  $C(S_i) \leftarrow 0$ 
16:    end for
17:    for each terminal trajectory  $T \in \mathcal{T}_q$  do
18:      Compute  $R(T) = R_{\text{out}} + R_{\text{fmt}} + R_{\text{pen}}$ 
19:      for each prefix snapshot  $S_i \preceq T$  do
20:         $R_{\text{acc}}(S_i) \leftarrow R_{\text{acc}}(S_i) + R(T)$ 
21:         $C(S_i) \leftarrow C(S_i) + 1$ 
22:      end for
23:    end for
24:    for each snapshot  $S_i \in \mathcal{V}_q$  do
25:      Set  $R(S_i) \leftarrow R_{\text{acc}}(S_i) / C(S_i)$ 
26:    end for
27:  end for
28:  Group snapshots by query and compute  $\hat{A}_i$  by Eq. 9
29:  for GRPO update epoch  $= 1, \dots, \mu$  do
30:    Update  $\pi_\theta$  with the GRPO objective, treating each snapshot as an independent sample
31:  end for
32: end for

```

Here, R_{out} is computed by comparing the model’s final answer with the ground truth, while R_{fmt} checks whether the model generates tool calls instead of plain text. The penalty term R_{pen} penalizes failed tool executions, such as calling readMemory before any memory has been built or context-length violations. **For an intermediate trajectory snapshot** S_i , its reward is determined by all terminal trajectories that take S_i as a prefix:

$$R(S_i) = \frac{1}{|\mathcal{T}(S_i)|} \sum_{T \in \mathcal{T}(S_i)} R(T) \quad (8)$$

where $\mathcal{T}(S_i)$ denotes the set of terminal trajectories with S_i as a prefix. After obtaining snapshot-level rewards, we group all trajectory snapshots generated under the same query q as an advantage calcu-

lation group $\mathcal{G} = \{S_t^{(j)} \mid \forall j, t\}$. We then compute the advantage of each snapshot sample $S_t^{(j)}$ using the group mean and standard deviation:

$$\hat{A}_t^{(j)} = \frac{R(S_t^{(j)}) - \text{mean}(\{R(S) \mid S \in \mathcal{G}\})}{\text{std}(\{R(S) \mid S \in \mathcal{G}\})} \quad (9)$$

Finally, we optimize π_θ with the GRPO objective (Shao et al., 2024), treating each trajectory snapshot as an independent sample. This shifts credit assignment from trajectories to snapshots, enabling more precise reward estimation. Appendix A theoretically analyzes the benefit from a variance-reduction perspective.

5 Experiments

5.1 Experiment Setup

Base Models and Training Data. To evaluate the effectiveness of *ContextPilot*, we conduct extensive experiments on two long-horizon tasks: long-context QA and deep search. **For long-context QA**, we follow the data construction setup of StateLM (Liu et al., 2026b). Specifically, we construct SFT data from the PublicDomain split of NovelQA (Wang et al., 2025) and the training split of NarrativeQA (Kočiský et al., 2018), and perform reinforcement learning on the training set of LongBench-v2 (Bai et al., 2025). For model selection, we use Qwen3-8B, Qwen3-14B (Yang et al., 2025) and Gemma4-E4B-it (Google, 2026) as base models. **For deep search**, base models are WebSailor-7B (Li et al., 2025a) and WebExplorer-8B (Liu et al., 2025). Since they already possess basic search capabilities, we skip SFT and directly perform RL training on 1K samples drawn from OpenSeeker (Du et al., 2026). Detailed statistics of training data are provided in Appendix Table 7.

Implementation Details. We conduct SFT and RL training with the verl library (Sheng et al., 2025). In RL, we limit each complete trajectory to be segmented into at most 8 trajectory snapshots. For each query, we collect $N = 128$ trajectory snapshots, starting from 8 trajectory-level rollouts, producing at most $M = 64$ trajectory snapshots, and completing the remaining budget with partial rollout. During inference, we set the maximum input and generation lengths to 30K and 2K tokens, respectively. More details are in Appendix D.

Evaluation Benchmarks and Metrics. We evaluate long-context QA on four benchmarks: the

Copyright split of NovelQA (Wang et al., 2025), the En.MC split of ∞ Bench (Zhang et al., 2024), LongMemEval-S (Wu et al., 2025a), and BrowseComp+ (Chen et al., 2025). Note that BrowseComp+ is built on a fixed corpus and does not require searching on the Internet. We therefore include it in the long-context QA tasks. The evaluation on deep search tasks includes GAIA (the text-only subset with 103 examples) (Mialon et al., 2024), BrowseComp (Wei et al., 2025), BrowseComp-ZH (Zhou et al., 2025), and xBench-DeepSearch (XBenchTeam, 2025). We adopt exact-match evaluation for multiple-choice benchmarks, NovelQA and ∞ Bench. Other benchmarks are evaluated by LLM-as-a-Judge, where GPT-OSS-120B (OpenAI, 2025) serves as the judge model.

Baselines. **For long-context QA**, we compare *ContextPilot* with (1) a training-free method: ReadAgent (Lee et al., 2024), (2) an RL-trained memory agent: MemAgent (Yu et al., 2025), (3) a proactive context management agent: StateLM (Liu et al., 2026b), and (4) a prompt-only baseline that uses the same tools as *ContextPilot* but without any fine-tuning, denoted as “w/ tools”. **Regarding deep search**, we include the following baselines: (1) ReAct (w/ truncation), which truncates early messages when the context exceeds 28K tokens, (2) ReSum (Wu et al., 2025b), an inference-time summarization method, (3) SUPO (Lu et al., 2025), which jointly trains summarization and agentic ability through RL, and (4) OpenSeeker, which performs RL training on the same 1K samples as ours but without context management tools. Baseline replication details are in Appendix M.

5.2 Main Results

Tables 2 and 3 compare *ContextPilot* with various baselines on long-context QA and deep search tasks. We highlight three key observations.

(1) *ContextPilot* achieves the best average performance among comparable-size models. As shown in Table 2, *ContextPilot* uses only a 32K context window yet outperforms the 128K backbone. It also achieves clear gains over prior RL-trained context management agents. For example, ContextPilot-8B-RL exceeds StateLM-8B-RL by 3.55 average points on long-context QA, while ContextPilot surpasses SUPO by 2.54 average points on WebSailor-7B for deep search.

(2) RL training brings more pronounced gains in more complex long-context settings. Starting

Table 2: Performance comparison of *ContextPilot* against baselines on long-context QA tasks. For *ContextPilot*, we run three times and report the mean and standard deviation. Results with [†] are from Liu et al. (2026b).

Model	Length	NovelQA	∞ Bench	LongMemEval-S	BrowseComp+	Avg.
Qwen3.5-397B-A17B (w/o tools)	256K	88.77	90.39	81.00	62.05	80.55
Qwen3.5-397B-A17B (w/ tools)	32K	91.94	92.13	83.60	80.96	87.16
RL-MemoryAgent-7B [†]	32K	60.24	62.45	40.60	-	-
RL-MemoryAgent-14B [†]	32K	78.86	74.24	59.00	-	-
ReadAgent-8B [†]	32K	16.38	24.02	0.00	-	-
ReadAgent-14B [†]	32K	23.12	34.06	14.60	-	-
StateLM-8B-RL [†]	32K	84.15 $\pm$ 1.00	73.07 $\pm$ 1.33	59.73 $\pm$ 2.20	46.44 $\pm$ 0.77	65.85
StateLM-14B-RL [†]	32K	84.85 $\pm$ 0.42	78.46 $\pm$ 0.67	64.47 $\pm$ 0.50	52.67 $\pm$ 4.00	70.11
Qwen3-8B (w/o tools)	128K	65.74 $\pm$ 0.55	66.96 $\pm$ 1.09	45.20 $\pm$ 1.02	5.82 $\pm$ 0.86	45.93
Qwen3-8B (w/ tools)	32K	38.09 $\pm$ 0.88	39.59 $\pm$ 1.03	24.47 $\pm$ 0.57	8.28 $\pm$ 0.11	27.61
ContextPilot-8B	32K	82.56 $\pm$ 0.49	71.03 $\pm$ 1.44	60.67 $\pm$ 1.91	48.84 $\pm$ 1.48	65.78
ContextPilot-8B-RL	32K	83.88 $\pm$ 0.67	75.25 $\pm$ 0.82	64.27 $\pm$ 1.15	54.18 $\pm$ 1.47	69.40
Qwen3-14B (w/o tools)	128K	78.03 $\pm$ 0.44	74.53 $\pm$ 0.41	54.20 $\pm$ 0.75	6.27 $\pm$ 0.87	53.26
Qwen3-14B (w/ tools)	32K	68.43 $\pm$ 1.22	54.59 $\pm$ 0.94	40.33 $\pm$ 0.50	15.78 $\pm$ 0.10	44.78
ContextPilot-14B	32K	84.28 $\pm$ 0.76	79.04 $\pm$ 0.94	65.93 $\pm$ 1.20	53.13 $\pm$ 1.37	70.60
ContextPilot-14B-RL	32K	84.81 $\pm$ 0.86	81.08 $\pm$ 1.15	67.40 $\pm$ 0.91	55.50 $\pm$ 1.71	72.20
Gemma4-E4B-it (w/o tools)	128K	48.75 $\pm$ 0.53	39.74 $\pm$ 0.87	28.50 $\pm$ 1.90	7.03 $\pm$ 0.30	31.01
Gemma4-E4B-it (w/ tools)	32K	36.90 $\pm$ 2.06	32.75 $\pm$ 2.17	21.80 $\pm$ 1.30	2.73 $\pm$ 0.30	23.55
ContextPilot-E4B	32K	66.80 $\pm$ 0.81	55.02 $\pm$ 1.07	55.07 $\pm$ 1.20	42.05 $\pm$ 1.04	54.74
ContextPilot-E4B-RL	32K	72.92 $\pm$ 0.86	60.99 $\pm$ 1.25	62.47 $\pm$ 1.62	47.47 $\pm$ 1.08	60.96

Table 3: Performance comparison of *ContextPilot* against baseline methods on deep search tasks. We run each method three times and report the mean and standard deviation.

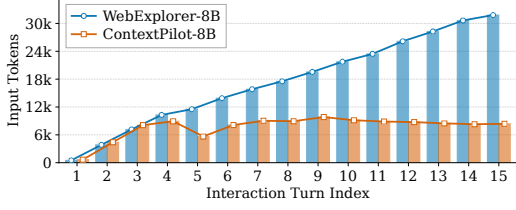
Backbone	Method	BrowseComp pass@3	BrowseComp-ZH pass@3	GAIA pass@1	xBench-DS pass@1	Avg.
WebSailor-7B	ReAct	11.33 $\pm$ 1.03	25.47 $\pm$ 0.98	31.07 $\pm$ 0.79	34.00 $\pm$ 0.82	25.47
	ReAct (w/ truncation)	12.67 $\pm$ 0.62	27.91 $\pm$ 0.91	33.66 $\pm$ 0.92	35.00 $\pm$ 0.82	27.31
	ReSum	15.83 $\pm$ 0.85	38.99 $\pm$ 1.56	38.19 $\pm$ 1.21	35.33 $\pm$ 1.25	32.09
	SUPO	18.50 $\pm$ 1.08	42.68 $\pm$ 0.99	42.07 $\pm$ 1.65	42.00 $\pm$ 1.41	36.31
	OpenSeeker	16.50 $\pm$ 2.12	41.87 $\pm$ 1.02	41.42 $\pm$ 1.21	43.33 $\pm$ 0.94	35.78
	ContextPilot	21.17 $\pm$ 1.31	43.14 $\pm$ 1.07	45.31 $\pm$ 0.92	43.67 $\pm$ 0.94	38.32
WebExplorer-8B	ReAct	23.83 $\pm$ 1.55	46.57 $\pm$ 0.74	48.87 $\pm$ 0.92	52.33 $\pm$ 0.94	42.90
	ReAct (w/ truncation)	24.33 $\pm$ 1.18	45.91 $\pm$ 1.07	49.84 $\pm$ 0.46	52.33 $\pm$ 0.47	43.10
	ReSum	28.83 $\pm$ 1.03	47.87 $\pm$ 1.39	52.75 $\pm$ 1.21	51.00 $\pm$ 1.41	45.11
	SUPO	31.00 $\pm$ 1.78	50.40 $\pm$ 1.07	56.96 $\pm$ 1.21	58.00 $\pm$ 0.82	49.09
	OpenSeeker	29.17 $\pm$ 1.31	48.56 $\pm$ 1.39	57.28 $\pm$ 1.37	56.67 $\pm$ 0.47	47.92
	ContextPilot	32.17 $\pm$ 1.18	53.63 $\pm$ 1.23	57.93 $\pm$ 0.92	56.67 $\pm$ 0.94	50.10

from the SFT model, RL further improves *ContextPilot*. For instance, ContextPilot-8B-RL improves over ContextPilot-8B by an average of 3.62 points across the four long-context QA tasks. The improvement on NovelQA is relatively modest, because the SFT data already includes another split of NovelQA. By contrast, RL yields a substantial 5.34 point increase on the more challenging BrowseComp+ benchmark. Considering the difficulty gap between NovelQA and BrowseComp+, this contrast suggests that the benefit of RL is larger on more challenging tasks.

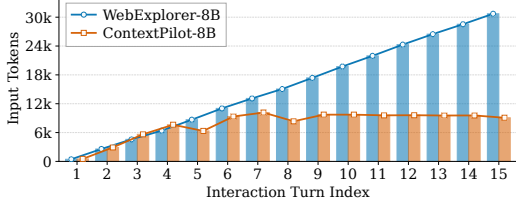
(3) The advantages of *ContextPilot* generalize across tasks and base models. As shown in Tables 2 and 3, *ContextPilot* brings consistent gains on both long-context QA and deep search. The gains also hold across different backbones, including Qwen3-8B, Qwen3-14B and Gemma4-E4B on long-context QA, as well as WebSailor-7B and WebExplorer-8B on deep search.

5.3 Further Analysis

Token efficiency analysis. Beyond performance, we examine whether *ContextPilot* can maintain a



(a) BrowseComp.



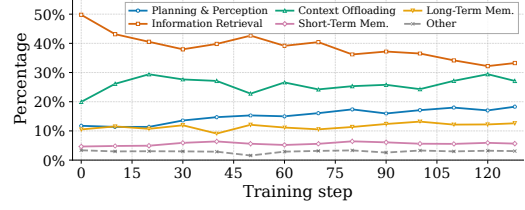
(b) BrowseComp-ZH.

Figure 4: Token usage per turn on deep search tasks.

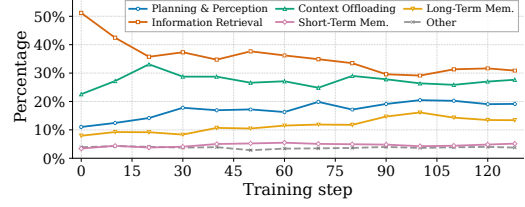
more compact working context. We consider trajectories with at least 15 interaction turns and compute the average number of input tokens per turn. As shown in Figure 4, the input length of WebExplorer-8B grows almost linearly on BrowseComp, reaching around 30K tokens. By contrast, ContextPilot-8B stabilizes its input length each turn at roughly 8K–10K tokens. This trend shows that *ContextPilot* can retain useful information while pruning redundant history, enabling stronger performance with substantially lower context overhead.

RL reshapes tool use strategy. To better understand how RL changes the context management behavior, we track the distribution over tool call categories of ContextPilot-8B-RL in RL training. As shown in Figure 5, the model relies heavily on information retrieval tools in the early stage of RL training, with retrieval accounting for roughly half of all tool calls. As RL training proceeds, the share of information retrieval tools gradually decreases, whereas planning and perception, long-term memory, and context offloading tools exhibit an upward trend. This shift suggests that RL encourages the model to move beyond naive information retrieval toward more coordinated context management.

RL synergistically boosts tool-use correctness and task success. Beyond the tool call frequency breakdown, we further examine whether RL improves the correctness of context management tool use. To quantify this effect, we count tool invocation failures (defined in Appendix G) for each intermediate checkpoint of ContextPilot-8B-RL. Figure 6 reports the failure rates of four representa-



(a) ∞ Bench.



(b) NovelQA.

Figure 5: Tool category breakdown during Qwen3-8B RL training on ∞ Bench and NovelQA.

Table 4: Ablation results of tool design using Qwen3.5-397B-A17B. LME-S and BC+ denote LongMemEval-S and BrowseComp+, respectively.

Tool Design	NovelQA	∞ Bench	LME-S	BC+	Avg.
Original tools	88.90	85.15	74.00	63.49	77.89
+ Planning	89.76	87.23	78.50	65.66	80.29
+ Soft offloading	91.28	89.63	80.20	71.20	83.08
+ Long-term memory	91.94	92.13	83.60	80.96	87.16

tive tool categories together with the task success rate throughout RL training. At the early stage of training, memory and context offloading tools exhibit substantially higher failure rates than information retrieval tools. This pattern suggests that although the SFT model can invoke these tools, it lacks a genuine understanding of their usage. During RL training, the failure rates of memory-related and context offloading tools decrease substantially as the task success rate improves. These results indicate that improved proficiency with context management tools synergistically contributes to the overall performance gains.

5.4 Ablation Studies

Tool design. To isolate the effect of tool design, we evaluate Qwen3.5-397B-A17B without fine-tuning, using cumulative tool configurations that progressively add planning, soft context offloading, and long-term memory tools. As presented in Table 4, the performance improves as more context management tools are introduced, with the full toolset achieving the highest score. The gains are especially clear on BrowseComp+, where ac-

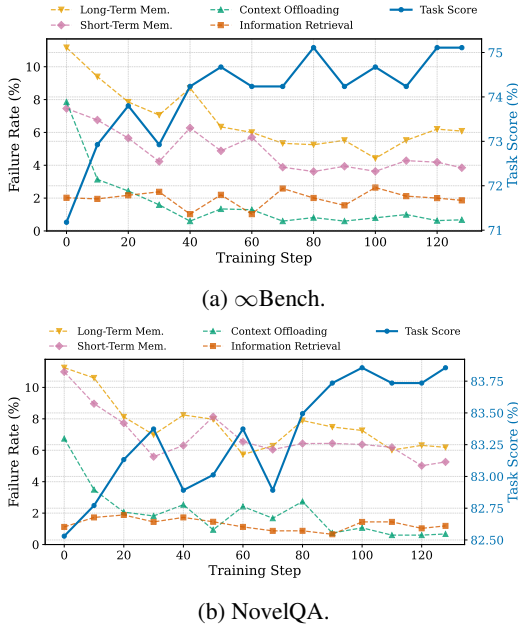


Figure 6: Tool invocation failure rates and task success rate on ∞ Bench and NovelQA during RL training.

Table 5: Ablation results of the RL algorithm using Qwen3-8B. “+ Entropy.”, “+ Context.”, and “+ Fine-grained.” denote RL training with entropy-based partial rollout, context-aware partial rollout, and fine-grained credit assignment. Rows marked “+” cumulatively build on the preceding row; parentheses indicate the corresponding changes.

Training method	NovelQA	∞ Bench	LME-S	BC+
SFT	82.56	71.03	60.67	48.84
GRPO	83.53 (+0.97)	72.78 (+1.75)	60.07 (-0.60)	50.96 (+2.12)
+ Entropy.	82.52 (-1.01)	73.07 (+0.29)	62.13 (+2.06)	49.64 (-1.32)
+ Context.	83.05 (+0.53)	73.94 (+0.87)	61.40 (-0.73)	51.08 (+1.44)
+ Fine-grained.	83.88 (+0.83)	75.25 (+1.31)	64.27 (+2.87)	54.18 (+3.10)

curacy increases from 63.49% to 80.96% with the full toolset, highlighting the effectiveness of the newly introduced tools. Appendix I further shows that the gains from our extended toolset transfer to multiple strong backbone models without task-specific fine-tuning. Table 6 presents a detailed ablation of toolset design at both the SFT and RL stages on Qwen3-8B. Compared with the original toolset, our extended toolset improves the average score by 1.56 points after SFT and 2.18 points after RL. These results disentangle the two sources of improvement, confirming that the extended toolset and RL training make distinct, complementary contributions to the final performance.

RL training design. We further ablate context-aware partial rollout and fine-grained credit assignment in RL training. As shown in Table 5, entropy-based partial rollout improves several tasks but

Table 6: Ablation results of toolset design at the SFT and RL stages on Qwen3-8B.

Toolset	NovelQA	∞ Bench	LME-S	BC+	Avg.
SFT					
Original	82.03	69.00	59.07	46.78	64.22
Ours	82.56	71.03	60.67	48.84	65.78 (+1.56)
RL					
Original	84.19	72.05	61.27	51.37	67.22
Ours	83.88	75.25	64.27	54.18	69.40 (+2.18)

remains unstable, decreasing BrowseComp+ accuracy by 1.32 points on Qwen3-8B. In addition, context variation alone is insufficient as a branching signal for partial rollout, as shown in Table 11 (Appendix). These results suggest that neither signal alone reliably identifies critical context-editing operations. Combining context variation and generation entropy into a joint sensitivity score yields the strongest partial rollout result. Fine-grained credit assignment further improves over context-aware partial rollout across all four benchmarks, raising the average score by 2.03 points, with an especially large gain of 3.10 points on the more challenging BrowseComp+ task. These results highlight the importance of fine-grained credit assignment.

6 Conclusion

We present *ContextPilot*, a proactive context management agent system for agentic reasoning. It extends the context management toolset with planning, long-term memory, and soft context offloading tools, and further improves RL training with context-aware partial rollout and fine-grained credit assignment. Experiments show that *ContextPilot* maintains a more compact working context while achieving stronger performance, bringing consistent gains on both long-context QA and deep search tasks. These results highlight the importance of proactive context management and suggest a promising path toward stronger long-horizon agents that can manage their own context.

Limitations

While *ContextPilot* demonstrates strong performance and improved token efficiency, this work has several limitations. First, although our approach extends existing context management tools, the toolset may still not cover all forms of context editing demands. Future work can explore richer operations for organizing, compressing, and retrieving context under different task requirements. Second,

due to computational constraints, we do not conduct extensive search over some training hyperparameters. These settings may affect training efficiency and final performance, especially for partial rollout and credit assignment. Finally, our experiments focus mainly on long-context QA and deep search tasks. Extending proactive context management to broader agentic reasoning scenarios, such as agentic coding and GUI agents, remains an important direction for future work.

References

- Yushi Bai, Shangqing Tu, Jiajie Zhang, Hao Peng, Xiaozhi Wang, Xin Lv, Shulin Cao, Jiazheng Xu, Lei Hou, Yuxiao Dong, and 1 others. 2025. Longbench v2: Towards deeper understanding and reasoning on realistic long-context multitasks. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 3639–3664.
- Zijian Chen, Xueguang Ma, Shengyao Zhuang, Ping Nie, Kai Zou, Andrew Liu, Joshua Green, Kshama Patel, Ruoxi Meng, Mingyi Su, and 1 others. 2025. Browsecomp-plus: A more fair and transparent evaluation benchmark of deep-research agent. *arXiv preprint arXiv:2508.06600*.
- Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. 2025. Mem0: Building production-ready ai agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*.
- Guanting Dong, Hangyu Mao, Kai Ma, Licheng Bao, Yifei Chen, Zhongyuan Wang, Zhongxia Chen, Jiazheng Du, Huiyang Wang, Fuzheng Zhang, and 1 others. 2025. Agentic reinforced policy optimization. *arXiv preprint arXiv:2507.19849*.
- Bangde Du, Ziyi Ye, Zhijing Wu, Monika A Jankowska, Shuqi Zhu, Qingyao Ai, Yujia Zhou, and Yiqun Liu. 2025. Simvbg: Simulating individual values by back-story generation. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 13104–13133.
- Yuwen Du, Rui Ye, Shuo Tang, Xinyu Zhu, Yijun Lu, Yuzhu Cai, and Siheng Chen. 2026. Openseeker: Democratizing frontier search agents by fully open-sourcing training data. *arXiv preprint arXiv:2603.15594*.
- Jiazheng Feng, Shijue Huang, Xingwei Qu, Ge Zhang, Yujia Qin, Baoquan Zhong, Chengquan Jiang, Jinxin Chi, and Wanjuan Zhong. 2025. Retool: Reinforcement learning for strategic tool use in llms. *arXiv preprint arXiv:2504.11536*.
- Google. 2026. Welcome Gemma 4: Frontier multimodal intelligence on device. <https://huggingface.co/blog/gemma4>.
- Tomáš Kočiský, Jonathan Schwarz, Phil Blunsom, Chris Dyer, Karl Moritz Hermann, Gábor Melis, and Edward Grefenstette. 2018. The narrativeqa reading comprehension challenge. *Transactions of the Association for Computational Linguistics*, 6:317–328.
- Kuang-Huei Lee, Xinyun Chen, Hiroki Furuta, John Canny, and Ian Fischer. 2024. A human-inspired reading agent with gist memory of very long contexts. In *International Conference on Machine Learning*, pages 26396–26415. PMLR.
- Jiazheng Li, Yawei Wang, Qiaojing Yan, Yijun Tian, Zhichao Xu, Huan Song, Panpan Xu, and Lin Lee Cheong. 2026. Salt: Step-level advantage assignment for long-horizon agents via trajectory graph. In *Findings of the Association for Computational Linguistics: EACL 2026*, pages 4709–4725.
- Kuan Li, Zhongwang Zhang, Huifeng Yin, Liwen Zhang, Litu Ou, Jialong Wu, Wenbiao Yin, Baixuan Li, Zhengwei Tao, Xinyu Wang, Weizhou Shen, Junkai Zhang, Dingchu Zhang, Xixi Wu, Yong Jiang, Ming Yan, Pengjun Xie, Fei Huang, and Jingren Zhou. 2025a. Websailor: Navigating super-human reasoning for web agent. *arXiv preprint arXiv:2507.02592*.
- Mo Li, LH Xu, Qitai Tan, Long Ma, Ting Cao, and Yunxin Liu. 2025b. Sculptor: Empowering llms with cognitive agency via active context management. *arXiv preprint arXiv:2508.04664*.
- Junteng Liu, Yunji Li, Chi Zhang, Jingyang Li, Aili Chen, Ke Ji, Weiyu Cheng, Zijia Wu, Chengyu Du, Qidi Xu, Jiayuan Song, Zhengmao Zhu, Wenhui Chen, Pengyu Zhao, and Junxian He. 2025. Webexplorer: Explore and evolve for training long-horizon web agents. *ArXiv*, abs/2509.06501.
- Shuyang Liu, Saman Dehghan, Jatin Ganhotra, Martin Hirzel, and Reyhaneh Jabbarvand. 2026a. From plan to action: How well do agents follow the plan? *arXiv preprint arXiv:2604.12147*.
- Xiaoyuan Liu, Tian Liang, Dongyang Ma, Deyu Zhou, Haitao Mi, Pinjia He, and Yan Wang. 2026b. The pensieve paradigm: Stateful language models mastering their own context. In *The Fourteenth International Conference on Learning Representations*.
- Junru Lu, Siyu An, Mingbao Lin, Gabriele Pergola, Yulan He, Di Yin, Xing Sun, and Yunsheng Wu. 2023. Memochat: Tuning llms to use memos for consistent long-range open-domain conversation. *arXiv preprint arXiv:2308.08239*.
- Miao Lu, Weiwei Sun, Weihua Du, Zhan Ling, Xuesong Yao, Kang Liu, and Jiecao Chen. 2025. Scaling llm multi-turn rl with end-to-end summarization-based context management. *arXiv preprint arXiv:2510.06727*.
- Zhengxuan Lu, Dongfang Li, Yukun Shi, Beilun Wang, Longyue Wang, and Baotian Hu. 2026. Structured episodic event memory. *arXiv preprint arXiv:2601.06411*.

- Grégoire Mialon, Clémentine Fourier, Thomas Wolf, Yann LeCun, and Thomas Scialom. 2024. Gaia: a benchmark for general ai assistants. In *The Twelfth International Conference on Learning Representations*.
- OpenAI. 2025. [gpt-oss-120b & gpt-oss-20b model card](#). Preprint, arXiv:2508.10925.
- Charles Packer, Vivian Fang, Shishir G Patil, Kevin Lin, Sarah Wooders, and Joseph E Gonzalez. 2024. Memgpt: Towards llms as operating systems. In *The Twelfth International Conference on Learning Representations*.
- Zhuoshi Pan, Qianhui Wu, Huiqiang Jiang, Xufang Luo, Hao Cheng, Dongsheng Li, Yuqing Yang, Chin-Yew Lin, H Vicky Zhao, Lili Qiu, and 1 others. 2025. Secom: On memory construction and retrieval for personalized conversational agents. In *The Thirteenth International Conference on Learning Representations*.
- Zhuoshi Pan, Qianhui Wu, Huiqiang Jiang, Menglin Xia, Xufang Luo, Jue Zhang, Qingwei Lin, Victor Rühle, Yuqing Yang, Chin-Yew Lin, and 1 others. 2024. Llmilingua-2: Data distillation for efficient and faithful task-agnostic prompt compression. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 963–981.
- Jiaqi Shao, Yufeng Miao, Wei Zhang, and Bing Luo. 2025. Foldact: Efficient and stable context folding for long-horizon search agents. *arXiv preprint arXiv:2512.22733*.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Yang Wu, and 1 others. 2024. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*.
- Guangming Sheng, Chi Zhang, Zilinfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. 2025. Hybridflow: A flexible and efficient rlhf framework. In *Proceedings of the Twentieth European Conference on Computer Systems*, pages 1279–1297.
- Jin Su, Runnan Fang, Yeqiu Li, Xiaobin Wang, Shihao Cai, Pengjun Xie, Ningyu Zhang, and Fajie Yuan. 2026. U-fold: Dynamic intent-aware context folding for user-centric agents. *arXiv preprint arXiv:2601.18285*.
- Qwen Team. 2026. [Qwen3.5: Accelerating productivity with native multimodal agents](#).
- Tongyi DeepResearch Team, Baixuan Li, Bo Zhang, Dingchu Zhang, Fei Huang, Guangyu Li, Guoxin Chen, Huifeng Yin, Jialong Wu, Jingren Zhou, and 1 others. 2025. Tongyi deepresearch technical report. *arXiv preprint arXiv:2510.24701*.
- Cunxiang Wang, Ruoxi Ning, Boqi Pan, Tonghui Wu, Qipeng Guo, Cheng Deng, Guangsheng Bao, Xiangukun Hu, Zheng Zhang, Qian Wang, and 1 others. 2025. Novelqa: Benchmarking question answering on documents exceeding 200k tokens. In *The Thirteenth International Conference on Learning Representations*.
- Jason Wei, Zhiqing Sun, Spencer Papay, Scott McKinney, Jeffrey Han, Isa Fulford, Hyung Won Chung, Alex Tachard Passos, William Fedus, and Amelia Glaese. 2025. Browsecomp: A simple yet challenging benchmark for browsing agents. *arXiv preprint arXiv:2504.12516*.
- Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. 2025a. Longmemeval: Benchmarking chat assistants on long-term interactive memory. In *The Thirteenth International Conference on Learning Representations*.
- Xixi Wu, Kuan Li, Yida Zhao, Liwen Zhang, Litu Ou, Huifeng Yin, Zhongwang Zhang, Yong Jiang, Pengjun Xie, Fei Huang, Minhao Cheng, Shuai Wang, Hong Cheng, and Jingren Zhou. 2025b. Resum: Unlocking long-horizon search intelligence via context summarization. *arXiv preprint arXiv:2509.13313*.
- Xbench-Team. 2025. [Xbench-deepsearch](#).
- Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. 2026. A-mem: Agentic memory for llm agents. *Advances in Neural Information Processing Systems*, 38:17577–17604.
- An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, and 1 others. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. React: Synergizing reasoning and acting in language models. In *The Eleventh International Conference on Learning Representations*.
- Rui Ye, Zhongwang Zhang, Kuan Li, Huifeng Yin, Zhengwei Tao, Yida Zhao, Liangcai Su, Liwen Zhang, Zile Qiao, Xinyu Wang, and 1 others. 2025. Agentfold: Long-horizon web agents with proactive context management. *arXiv preprint arXiv:2510.24699*.
- Hongli Yu, Tinghong Chen, Jiangtao Feng, Jiangjie Chen, Weinan Dai, Qiying Yu, Ya-Qin Zhang, Wei-Ying Ma, Jingjing Liu, Mingxuan Wang, and Hao Zhou. 2025. Memagent: Reshaping long-context llm with multi-conv rl-based memory agent. *arXiv preprint arXiv:2507.02259*.
- Yi Yu, Liuyi Yao, Yuexiang Xie, Qingquan Tan, Jiaqi Feng, Yaliang Li, and Libing Wu. 2026. Agentic memory: Learning unified long-term and short-term memory management for large language model agents. *arXiv preprint arXiv:2601.01885*.

Xinrong Zhang, Yingfa Chen, Shengding Hu, Zihang Xu, Junhao Chen, Moo Hao, Xu Han, Zhen Thai, Shuo Wang, Zhiyuan Liu, and 1 others. 2024. ∞ -bench: Extending long context evaluation beyond 100k tokens. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 15262–15277.

Yuxiang Zhang, Jiangming Shu, Ye Ma, Xueyuan Lin, Shangxi Wu, and Jitao Sang. 2025. Memory as action: Autonomous context curation for long-horizon agentic tasks. *arXiv preprint arXiv:2510.12635*.

Peilin Zhou, Bruce Leon, Xiang Ying, Can Zhang, Yifan Shao, Qichen Ye, Dading Chong, Zhiling Jin, Chenxuan Xie, Meng Cao, and 1 others. 2025. Browsecomp-zh: Benchmarking web browsing ability of large language models in chinese. *arXiv preprint arXiv:2504.19314*.

A Theoretical Discussion of Fine-Grained Credit Assignment

We justify our fine-grained credit assignment as a lower-variance estimator of the conditional continuation value. For a query q , let T be a complete trajectory with terminal reward $R(T)$, and let S be an intermediate trajectory snapshot induced by a context editing action. The target credit for S is

$$Q(S) \triangleq \mathbb{E}[R(T) \mid S \preceq T], \quad (10)$$

where $S \preceq T$ denotes that S is a prefix snapshot of T .

Trajectory-level credit assignment uses the reward of the single terminal trajectory containing S , while our method averages all sampled terminal continuations that pass through S :

$$\begin{aligned} \hat{Q}_{\text{traj}}(S) &\triangleq R(T_1), \\ \hat{Q}_{\text{ours}}(S) &\triangleq \frac{1}{n_S} \sum_{k=1}^{n_S} R(T_k). \end{aligned} \quad (11)$$

Here $n_S = |\mathcal{T}(S)|$, and each T_k is sampled from the same continuation distribution $p(\cdot \mid S)$.

Claim. Conditioned on S , suppose $\{R(T_k)\}_{k=1}^{n_S}$ are independent samples with finite conditional variance

$$\sigma^2(S) \triangleq \text{Var}[R(T) \mid S \preceq T]. \quad (12)$$

Then both estimators are unbiased:

$$\begin{aligned} \mathbb{E}[\hat{Q}_{\text{traj}}(S) \mid S] &= Q(S), \\ \mathbb{E}[\hat{Q}_{\text{ours}}(S) \mid S] &= Q(S), \end{aligned} \quad (13)$$

and their conditional variances satisfy

$$\begin{aligned} \text{Var}[\hat{Q}_{\text{traj}}(S) \mid S] &= \sigma^2(S), \\ \text{Var}[\hat{Q}_{\text{ours}}(S) \mid S] &= \frac{\sigma^2(S)}{n_S}. \end{aligned} \quad (14)$$

Therefore, when $n_S > 1$ and $\sigma^2(S) > 0$, our estimator has strictly smaller mean-squared error than trajectory-level credit assignment.

Proof. Since each continuation is sampled from $p(\cdot \mid S)$,

$$\begin{aligned} \mathbb{E}[\hat{Q}_{\text{traj}}(S) \mid S] &= \mathbb{E}[R(T_1) \mid S] \\ &= Q(S), \end{aligned} \quad (15)$$

and

$$\begin{aligned} \mathbb{E}[\hat{Q}_{\text{ours}}(S) \mid S] &= \frac{1}{n_S} \sum_{k=1}^{n_S} \mathbb{E}[R(T_k) \mid S] \\ &= Q(S). \end{aligned} \quad (16)$$

Thus both estimators are unbiased.

The trajectory-level estimator uses one continuation, so

$$\text{Var}[\hat{Q}_{\text{traj}}(S) \mid S] = \sigma^2(S). \quad (17)$$

For our estimator, conditional independence gives

$$\begin{aligned} \text{Var}[\hat{Q}_{\text{ours}}(S) \mid S] &= \text{Var}\left[\frac{1}{n_S} \sum_{k=1}^{n_S} R(T_k) \mid S\right] \\ &= \frac{1}{n_S^2} \sum_{k=1}^{n_S} \text{Var}[R(T_k) \mid S] \\ &= \frac{\sigma^2(S)}{n_S}. \end{aligned} \quad (18)$$

Since both estimators are unbiased, their mean-squared errors equal their variances. The claim follows. $\square$

Thus, our method estimates the same target credit $Q(S)$ as trajectory-level credit assignment with lower variance. Since GRPO computes relative advantages from these reward estimates within each query group, this lower-variance estimator provides a more stable advantage signal for optimizing context editing decisions. Context-aware partial rollout strengthens this effect by increasing n_S for sensitive context management actions.

B Training Dataset Statistics

Table 7 summarizes the training data used by *ContextPilot* across SFT and RL stages, for both experimental scenarios. For SFT, we construct trajectories from the PublicDomain split of NovelQA and the training split of NarrativeQA, and retain trajectories that satisfy the outcome and context management quality requirements. The retained trajectories are further segmented into trajectory snapshots at context editing operations, producing 51,469 snapshots in total. For RL, we use 488 LongBench-v2 questions for long-context QA and 1,000 questions randomly sampled from the OpenSeeker (Du et al., 2026) dataset for deep search.

C SFT Data Synthesis Details

We synthesize SFT trajectories for long-context QA only. The teacher is Qwen3.5-397B-A17B (Team, 2026) in thinking mode, decoded with temperature 0.6, top- p of 0.95, and a maximum output length of 4K tokens. To elicit valid context management behavior from the teacher, we use a context management harness that supplies tool definitions together with procedural guidance for managing long interaction histories. Rather than exposing the full toolset at every step, the harness dynamically presents only tools whose preconditions are satisfied by the current state. For instance, readMemory is unavailable before any memory has been written, while readChunk and readMultiChunks are withheld until searchContext has been invoked. The harness also implements correction and retry mechanisms for recoverable generation errors. When the teacher provides an invalid argument, such as a nonexistent message id or an already deleted message id for deleteContext, summarizeContext or compressContext, the harness returns a targeted hint and asks the teacher to retry the tool call. Similarly, if the teacher attempts to answer in plain text without invoking finish, the harness reminds it to submit the answer through the required tool interface. These auxiliary hints serve only as generation-time scaffolding and are excluded from the final trajectories. If a retry is triggered, we keep only the final successful attempt, preventing failed intermediate calls from being imitated.

After trajectory generation, we apply a three-stage filtering pipeline. First, we perform outcome-based filtering with exact match. For samples that are initially answered incorrectly, we allow two additional retries and retain the trajectory if any retry

Stage	Source	Questions	Trajectories	Snapshots
SFT	NovelQA	3,096	2,987	50,691
	NarrativeQA	100	81	778
RL	LongBench-v2	488	–	–
	OpenSeeker-v1 (sampled)	1,000	–	–

Table 7: Statistics of the training data used in *ContextPilot*, where NovelQA, NarrativeQA, and LongBench-v2 are used for long-context QA and OpenSeeker-v1 is used for deep search.

yields the correct answer. Second, we use GPT-OSS-120B (OpenAI, 2025) for process-based filtering, removing trajectories that exhibit improper context management behavior. Finally, we discard trajectories whose peak context length exceeds 32K tokens, ensuring that the retained demonstrations remain compatible with the target context budget. This procedure starts from 3,196 questions and retains 3,114 trajectories after outcome-based filtering. Process and peak-token filtering further remove 46 trajectories, leaving 3,068 qualified trajectories. These trajectories are then segmented at context editing operations, producing 51,469 SFT trajectory snapshots.

D Training Details

We conduct SFT and RL training with the verl library (Sheng et al., 2025). During SFT, we use ZeRO-3 parallelism, a global batch size of 128, a learning rate of 5×10^{-6} , a cosine learning-rate scheduler, and a warmup ratio of 0.03. For RL, we train for 128 steps with a rollout batch size of 16 and set the KL coefficient to 0.001. For each query, we first sample 8 trajectory-level rollouts. Each complete trajectory is segmented into at most 8 trajectory snapshots, yielding up to 64 snapshots from the initial rollouts. We collect 128 snapshots per query, and use partial rollout to complete the remaining snapshots. The terminal reward follows the decomposition in Section 4. The outcome reward $R_{\text{out}} \in \{0, 1\}$ is set to 1 if the answer submitted by the model matches the ground-truth answer, and 0 otherwise. The format reward $R_{\text{fmt}} \in \{-0.5, 0\}$ penalizes violations of the tool-call protocol: if the model emits plain text instead of a valid tool call at any turn, we set $R_{\text{fmt}} = -0.5$; otherwise, $R_{\text{fmt}} = 0$. The penalty term $R_{\text{pen}} \in \{-0.5, 0\}$ accounts for execution and budget violations. Specifically, $R_{\text{pen}} = -0.5$ if any tool call in the trajectory triggers an execution error or if the input length at any turn exceeds 26K tokens; otherwise, $R_{\text{pen}} = 0$. For the sensitivity

Long-Context Document QA System Prompt

```
You are an AI assistant specialized in long-context document QA. Use the currently available tools to find evidence in the
attached document, save useful findings, keep the context compact, and submit the final answer through `finish`.

## Rules
- Use only tools that are currently available in the API payload. If a tool is mentioned here but not currently available, do
  not call it.
- Keep reasoning before tool calls brief. Put durable facts into `memorize` / `updateMemory` or `note` / `updateNote` instead
  of long assistant prose.
- For `readNote` and `loadMemory`, the `key` must be copied exactly from the current `<external_note>` / `<external_memory>`
  lists. If the relevant list says "No notes recorded." or "No memories recorded.", do not call that reader.
- When cleaning context, use exact `[msg_id=N]` values from previous messages. Do not target the system message, the original
  user question, or messages already deleted/truncated/summarized/compressed/folded unless `restoreContext` is available
  and needed.
- Final answers must be submitted with `finish`; do not answer in plain text outside the tool call.
- For multiple-choice questions, map the evidence to the option by role, not by keyword overlap. For questions asking who "
  claims", "proposes", "says", "reveals", or does another action, identify the actor who performs that action, not a
  nearby narrator, escort, recipient, or person being discussed.
- For occurrence/count questions, do not infer a total from the first matching chunk or from top-k search coverage. Search at
  least one independent wording or morphological variant, enumerate distinct occurrences with their chunk/location
  evidence, deduplicate overlapping references, and only then map the verified count to an option.
- For earliest/latest/first/last, eventual outcome, character-change, or "where will" questions, one locally correct passage
  is not sufficient. Search for the subject with transition or chronology terms, inspect later/earlier occurrences as
  required, and explicitly resolve superseded plans, offers, beliefs, and outcomes before choosing an option.
- For relationship, identity, and action-attribution questions, preserve subject, object, speaker, and temporal qualifiers in
  memory. Before finishing, check that the saved relation answers the exact direction asked rather than a nearby inverse
  or associated fact.
- Search results may include `section_hint`, the nearest preceding chapter/book/part/volume heading tracked during chunking.
  Use that heading for section-location questions; never infer a chapter number from `chunk_id`.

## Workflow
1. Call `analyzeText` first. If `plan` is available before indexing, write a concise initial strategy. Then call `buildIndex`.
2. Use `searchEngine` for precise keywords, names, dates, titles, rare phrases. If a search returns no useful chunks, try
  different keywords or queries.
3. Make a `plan` after a successful search.
4. Read evidence with `readChunk` or `readMultiChunks` (respect the tool's chunk-count limit, usually at most 3 ids).
5. After reading, save useful findings with `memorize`, `updateMemory`, `note`, or `updateNote`.
6. After saving notes or memories, clean up bulky context whenever cleanup tools are available:
  - If there are 2 or more uncleaned successful search-result messages, clean search-result tool responses first. Target only
    the msg_ids of the tool responses of `searchEngine`.
  - If there are 3 or more uncleaned `plan` calls, clean old `plan` assistant messages. Target the assistant msg_ids that
    invoked `plan`, not the short tool responses.
  - Then prune the current `readChunk` / `readMultiChunks` tool result, and delete the assistant message that invoked `
    memorize` / `updateMemory` / `note` / `updateNote`.
  - Use `deleteContext` for irrelevant content, `truncateContext` for useful spans, `summarizeContext` for manual summaries,
    and `compressContext` for automatic compression.
  - Never apply these cleanup tools to the same message twice. If a msg_id has already been deleted, truncated, summarized,
    or compressed, do not target that msg_id again.
7. Before finishing, if `readNote` or `loadMemory` is available, review relevant saved entries using only keys listed in the
  current Available Notes/Memories sections.
8. Continue searching, reading, memorizing, cleaning, and reviewing until the evidence is sufficient; then call `finish` with
  a concise answer grounded in the document.
```

Figure 7: System prompt used by *ContextPilot* for long-context document QA.

score used to select partial-rollout branching points, we set $\alpha = 1$ and $\beta = 1$. Appendix L analyzes the robustness of the sensitivity score to different α values. The maximum input sequence length is 30K tokens, and the maximum output length is 2K tokens. During inference, a temperature of 0.7 and top- p of 0.8 are used for our model. The maximum interaction budget is 60 turns for long-context QA and 60 tool calls for deep search following Wu et al. (2025b). The system prompts used by *ContextPilot* for long-context document QA and deep search are shown in Figures 7 and 8, respectively. For LLM-as-a-Judge grading, we use GPT-OSS-120B (OpenAI, 2025) with the open-ended judging

prompt of StateLM (Liu et al., 2026b), as shown in Figure 9.

E Additional Token Efficiency Analysis

Figure 4 presents the token-efficiency analysis on both BrowseComp and BrowseComp-ZH. We consider trajectories with at least 15 interaction turns and report the average number of input tokens per turn. Across both benchmarks, the WebExplorer-8B agent accumulates increasingly long contexts as the interaction proceeds. In contrast, *ContextPilot* keeps the working context substantially more compact, indicating that its context management tools reduce redundant history without relying on

Deep Search System Prompt

You are a web-search QA agent. Answer the user's question with evidence gathered through the currently available tools, keep the conversation compact, and submit the final answer through `finish`.

Rules

- In search phase, use `search` for batched web queries, `visit` to read promising pages, and `finish` when you have found the answer.
- If `checkBudget` shows that the context length is beyond the threshold, call cleanup tools.
- When cleaning context, use exact `[msg\_id=N]` values. Do not target the system message, the original user question, or messages already deleted/truncated/summarized/compressed/folded.
- Final answers must be submitted with `finish`; do not answer in plain text outside the tool call.

Workflow

1. Use `search` with concise, complementary query strings to explore names, translations, aliases, dates, clue interpretations, and likely sources.
2. Use `visit` on the most promising URLs. Give each visit a precise goal describing what to extract or verify.
3. Iterate search and visit until you have enough evidence. Keep pre-tool reasoning brief and avoid copying large retrieved text into assistant messages.
4. If cleanup tools are available, reduce bulky history before continuing:
 - Prefer large `search` or `visit` results that are no longer needed verbatim.
 - Preserve facts still needed for the final answer: names, aliases, titles, dates, URLs, clue matches, and intermediate conclusions.
 - Use `deleteContext` for irrelevant content, `truncateContext` for useful spans, `summarizeContext` for concise faithful summaries, `compressContext` for automatic compression, and `foldHistory` when available to fold accumulated history.
5. Continue researching if evidence is incomplete; otherwise call `finish` with a concise, evidence-grounded answer.

Figure 8: System prompt used by *ContextPilot* for deep search tasks.

LLM-as-a-Judge Prompt

Given a problem, its correct answer, and a student's answer below, your task is to review the student's answer and determine if it is correct by comparing it to the correct answer. If the student's answer is incomplete or ambiguous, assume it is incorrect.

Problem
{problem}

Answer
{answer}

Student Answer
{mode_ans}

Please put your final answer (True or False) in `\boxed{}`. Specifically, if the student's answer is correct, the final answer should be `\boxed{True}`; otherwise, the final answer should be `\boxed{False}`.

Figure 9: Prompt template used by the LLM-as-a-Judge for evaluating open-ended question-answering, following StateLM (Liu et al., 2026b).

a larger context window.

F Tool Call Statistics

Figure 5 visualizes how the tool-category distribution changes during Qwen3-8B RL training on ∞ Bench and NovelQA. At the beginning of RL training, information retrieval accounts for a large fraction of tool calls, indicating that the model primarily seeks evidence through repeated search and

retrieval. As training proceeds, the share of information retrieval gradually decreases, while planning and perception, long-term memory, and context offloading tools become more frequent. This shift suggests that RL encourages the model to move from retrieval-centric behavior toward a more coordinated context management strategy.

G Tool Failure Definitions

We consider a tool invocation failed if its execution triggers an environment-side error. Concretely, this includes malformed calls, invalid arguments, and violations of tool-specific preconditions. For retrieval tools, returning no matched evidence is not counted as a failure if the query and index are valid. Table 8 summarizes the specific cases that may lead to tool execution errors in our analysis. Figure 6 reports the execution failure rate for different tool categories on ∞ Bench and NovelQA. Across both benchmarks, memory-related and context offloading tools have higher failure rates at the early stage of RL training, indicating that the SFT model can invoke these tools but has not yet fully learned their correct usage conditions. These failure rates decrease substantially during RL training, showing that RL improves the reliability of context management operations while reshaping the model's tool use strategy.

Table 8: Tool execution failure reason analysis.

Tool	Failure Reason
plan	The predicted summary is empty or unparseable.
searchContext	The search query is empty or malformed, or the model searches before a valid index has been built.
readChunk	The requested chunk id is out of range, deleted, or not associated with the current index.
readMultiChunks	Any requested chunk id is invalid, duplicated, deleted, or exceeds the allowed batch size.
note	The note content is empty, unparseable, or does not specify the value to be stored.
updateNote	The target note does not exist, or the update content is empty or unparseable.
readNote	The requested note does not exist.
memorize	The memory content is empty or unparseable (e.g., inconsistent with the memory storage schema).
updateMemory	The target memory item does not exist, or the update content is empty, unparseable, or inconsistent with the memory schema.
readMemory	The requested memory item does not exist.
deleteContext	The target message index does not exist, has already been deleted, or refers to a protected message (e.g., system prompt or user query).
summarizeContext	The target message does not exist, has already been offloaded, or the generated summary is unparseable.
compressContext	The target message does not exist, has already been offloaded, or specifies an invalid compression ratio.
foldHistory	There is no historical context to fold, or the generated summary and keywords are unparseable.

H Additional Toolset Ablation

To complement the tool design ablation in the main text, we further train Qwen3-8B with the original toolset under the same SFT/RL recipe. As shown in Table 6, the *ContextPilot* toolset consistently improves average performance over the original toolset after both SFT and RL. This result suggests that the gains from the expanded tool design transfer to the smaller trained backbone, rather than only appearing in the large teacher model.

I Training-Free Toolset Transfer

To isolate the contribution of tool design, we evaluate the original and *ContextPilot* toolsets on three strong backbones without task-specific fine-tuning. As shown in Table 9, the *ContextPilot* toolset consistently improves all three backbones, with average gains ranging from 8.30 to 9.27 points. These results demonstrate the effectiveness of the expanded tool interface independent of task-specific training. The matched comparison on the smaller Qwen3-8B model in Table 6 further demonstrates that these gains persist after both SFT and RL.

J General Tool-Use Robustness

Table 10: General tool-use evaluation results on ACEBench and τ^2 -Bench.

Backbone	ACEBench			τ^2 -Bench		
	Base	Ours	Δ	Base	Ours	Δ
Qwen3-8B	65.3	63.8	-1.5	35.9	35.4	-0.5
Qwen3-14B	70.8	69.7	-1.1	40.9	39.7	-1.2
Gemma4-E4B-it	70.9	70.2	-0.7	38.7	36.1	-2.6

Despite being specialized for long-horizon context management, *ContextPilot* largely preserves the general tool-use ability of its backbone. As shown in Table 10, the average performance drop are only 1.10 points on ACEBench and 1.43 points on τ^2 -Bench across three model families. These small and consistent differences across both benchmarks and all three model families indicate that the effect is not specific to a particular backbone or evaluation setting. Overall, specialized context-management training has a nearly negligible impact on general tool-use competence while delivering substantial gains on long-context QA and deep-search tasks.

K Additional RL Training Ablation

Table 11 compares four strategies for selecting branch points in partial rollout under a fixed roll-

Table 9: Evaluation of the original and *ContextPilot* toolsets on strong backbones without task-specific fine-tuning.

Backbone	Toolset	NovelQA	∞ Bench	LongMemEval-S	BrowseComp+	Avg.
Qwen3.5-397B	Original	88.90	85.15	74.00	63.49	77.89
	<i>ContextPilot</i>	91.94	92.13	83.60	80.96	87.16
GLM-5.1	Original	89.80	84.90	75.00	62.10	77.90
	<i>ContextPilot</i>	91.10	91.70	83.00	79.00	86.20
Kimi-K2.6	Original	91.10	84.70	74.90	65.60	79.10
	<i>ContextPilot</i>	91.30	93.00	85.20	81.30	87.70

Table 11: Additional ablation results of the RL algorithm using Qwen3-8B. All selection variants use the same snapshot budget and independently build on GRPO. Context & Entropy partial rollout combines context variation and generation-entropy variation. Fine-grained credit is added on top of this joint strategy.

Training method	NovelQA	∞ Bench	LongMemEval-S	BrowseComp+	Avg.
SFT	82.56	71.03	60.67	48.84	65.78
GRPO	83.53	72.78	60.07	50.96	66.84
GRPO + Random partial rollout	82.45	72.85	60.00	49.82	66.28
GRPO + Entropy-only partial rollout	82.52	73.07	62.13	49.64	66.84
GRPO + Context-only partial rollout	83.18	74.38	60.67	49.72	66.99
GRPO + Context & Entropy partial rollout	83.05	73.94	61.40	51.08	67.37
+ Fine-grained credit assignment	83.88	75.25	64.27	54.18	69.40

out budget: random selection, selection based on generation-entropy variation, selection using only context variation, and joint selection based on both signals. In addition, table 12 reports the RL training ablation results on both Qwen3-8B and Gemma4-E4B-it. The results show a consistent trend across the two models: context-aware partial rollout improves over entropy-only branching, and fine-grained credit assignment further strengthens the final average performance.

L Sensitivity Analysis of Partial Rollout

We further examine the robustness of the sensitivity score by varying α while fixing $\beta = 1$. As shown in Figure 10, the default setting $\alpha = 1$ achieves the best or near-best result across both Qwen3-8B and Gemma4-E4B-it. Performance remains stable around the default setting, suggesting that the proposed sensitivity score is not overly sensitive to the exact weight assigned to context variation.

M Baseline Reproduction Details

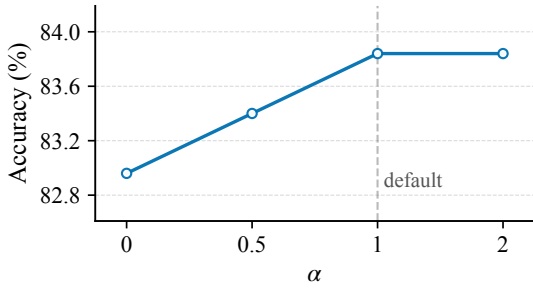
ReSum We reproduce the training-free version of ReSum (Wu et al., 2025b) for deep search, using WebSailor-7B and WebExplorer-8B as backbone

models. Following our deep search setting, all runs use a 32K context window, with 30K tokens allocated to the input and 2K tokens reserved for generation; summarization is triggered when the context exceeds 25K tokens, and the maximum tool call budget is set to 60. Since the official ReSum summarization model is not publicly available, we use Qwen3-30B-A3B as the external summarization model. For the summarizer, we adopt the recommended non-thinking decoding configuration of Qwen3, setting temperature to 0.7 and top- p to 0.8. The context summarization prompt and the summary-conditioned continuation prompt follow the original ReSum prompt.

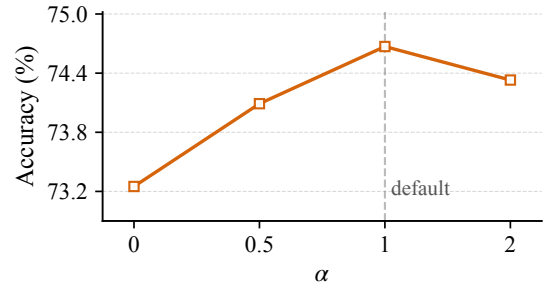
SUPO We reproduce SUPO (Lu et al., 2025) using WebSailor-7B and WebExplorer-8B as backbone models. To isolate the effect of the training data, we train SUPO on the same 1K OpenSeeker samples as *ContextPilot*. Following the design of SUPO, summaries are generated by the policy model itself and optimized jointly with tool use actions. The summarization threshold is set to $L = 0.95 \times 30K = 28.5K$ tokens, where 30K is our maximum input length. Following SUPO, we set the maximum number of interaction steps to

Table 12: Additional ablation results of the RL algorithm for Qwen3-8B and Gemma4-E4B-it. Rows marked “+” cumulatively build on the preceding row; parentheses indicate the corresponding changes.

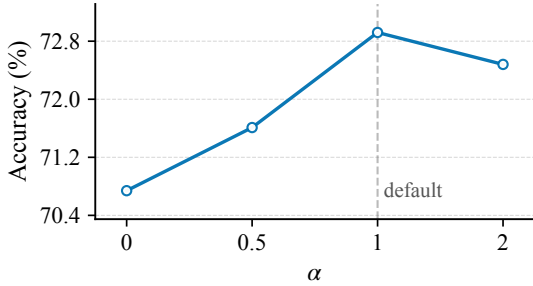
Model	Training method	NovelQA	∞ Bench	LongMemEval-S	BrowseComp+	Avg.
Qwen3-8B	SFT	82.56	71.03	60.67	48.84	65.78
	GRPO	83.53 (+0.97)	72.78 (+1.75)	60.07 (-0.60)	50.96 (+2.12)	66.84 (+1.06)
	+ Entropy-based partial rollout	82.52 (-1.01)	73.07 (+0.29)	62.13 (+2.06)	49.64 (-1.32)	66.84 (+0.00)
	+ Context-aware partial rollout	83.05 (+0.53)	73.94 (+0.87)	61.40 (-0.73)	51.08 (+1.44)	67.37 (+0.53)
	+ Fine-grained credit assignment	83.88 (+0.83)	75.25 (+1.31)	64.27 (+2.87)	54.18 (+3.10)	69.40 (+2.03)
Gemma4-E4B-it	SFT	66.80	55.02	55.07	42.05	54.74
	GRPO	71.12 (+4.32)	57.06 (+2.04)	57.00 (+1.93)	43.41 (+1.36)	57.15 (+2.41)
	+ Entropy-based partial rollout	70.81 (-0.31)	58.22 (+1.16)	60.67 (+3.67)	46.27 (+2.86)	58.99 (+1.85)
	+ Context-aware partial rollout	71.24 (+0.43)	59.68 (+1.46)	60.33 (-0.34)	46.14 (-0.13)	59.35 (+0.36)
	+ Fine-grained credit assignment	72.92 (+1.68)	60.99 (+1.31)	62.47 (+2.14)	47.47 (+1.33)	60.96 (+1.62)



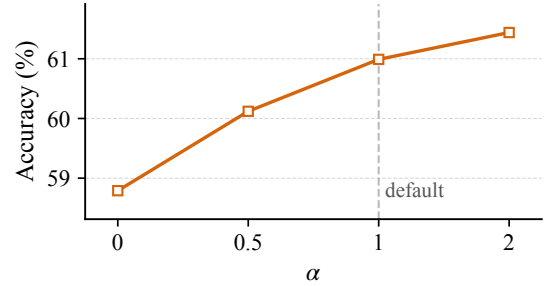
(a) Qwen3-8B on NovelQA.



(b) Qwen3-8B on ∞ Bench.



(c) Gemma4-E4B-it on NovelQA.



(d) Gemma4-E4B-it on ∞ Bench.

Figure 10: Sensitivity analysis of α in the context-aware partial rollout score with $\beta = 1$.

$H = 100$ and the maximum number of summaries to $S = 2$, and apply overlong masking to rollouts that fail to produce a final answer before reaching either limit. The remaining RL hyperparameters are also the same as SUPO: batch size $B = 32$, group size $G = 8$, learning rate 1×10^{-6} with a constant learning-rate scheduler, and clipping coefficients $\epsilon_{\text{high}} = 0.28$ and $\epsilon_{\text{low}} = 0.20$.

N Use of Scientific Artifacts

We cite the original creators of scientific artifacts wherever they are introduced. For models, Qwen3 and Gemma4 are released under the Apache License 2.0. For datasets, OpenSeeker-V1 is released under the MIT License, while the other datasets

used in this work are released under the Apache License 2.0. We use all artifacts only for research purposes and in a manner consistent with their intended use in benchmarking, model training, and evaluation. We do not redistribute the original datasets or model weights as part of this work. The artifacts cover two primary task domains: long-context QA and deep search. Language includes English and Chinese. We do not collect new data from human subjects. Because the experiments rely on existing public benchmarks and datasets, we do not perform additional manual anonymization or offensive-content filtering. Relevant data statistics, including the number of questions, retained trajectories, and trajectory snapshots used for training,

are reported in Table 7.

O Use of AI Assistants

We used Codex to assist with code modification, and ChatGPT for writing polishing. All assisted outputs were manually reviewed, verified, and finalized by the authors.